

# OpenClaw Client Guide

Multi-Agents Management · Python Client Reference

[Webhook Client](#)

[Tools Invoke](#)

[Tested](#)

呢份文件整理咗 `openclaw_client.py` 嘅用法、response schema、同步/非同步測試結果，同埋 project 入面 `test.py` 嘅實戰指令。

Prepared for Rex Chio · Generated by Bob ☐

## Overview

呢個 PDF 係靚仔版摘要＋完整技術文檔，方便 share 畀 team 睇，亦方便之後做 integration reference。

## Status

已完成實測：`wake()`、`list_sessions_details()`、`get_session_history_details()`、`call_agent()` `async`、`call_agent_sync()` `sync`。

# OPENCLAW\_CLIENT\_README

---

`openclaw_client.py` 係一個俾呢個 project 用嘅 OpenClaw Python client，主要包裝：

- Webhook endpoints
  - `POST /hooks/wake`
  - `POST /hooks/agent`
  - `POST /hooks/<name>`
- Tools invoke endpoint
  - `POST /tools/invoke`

呢份 README 會講：

- 點建立 client
- 點 call webhook / tools
- response format 係點
- `test.py` 點用
- 已經驗證過嘅行為同注意事項

## 1. 建立 client

```
import config
from openclaw_client import create_client

client = create_client(
    host=config.OPENCLAW_HOST,
    hooks_token=config.OPENCLAW_HOOKS_TOKEN,
    gateway_token=config.OPENCLAW_TOKEN,
    timeout=30,
)
```

### Config fields

- `host` : OpenClaw host，例如 `10.18.0.24:18789`
- `hooks_token` : webhook 用嘅 token
- `gateway_token` : `/tools/invoke` 用嘅 token
- `timeout` : HTTP request timeout，唔係 agent run timeout

## 2. Webhook methods

### wake()

```
result = client.wake("System check", mode="now")
```

用途：

- 觸發 main session heartbeat

參數：

- `text` : event 描述
- `mode` : `now` / `next-heartbeat`

### call\_agent()

```
result = client.call_agent(  
    message="Summarize inbox",  
    agent_id="main",  
    session_key="hook:main:test",  
    timeout_seconds=120,  
    deliver=False,  
)
```

用途：

- 觸發 isolated agent run

常用參數：

- `message` : 要 agent 處理嘅內容
- `name` : optional hook 名稱
- `agent_id` : 指定 agent
- `session_key` : 指定 session key
- `wake_mode` : `now` / `next-heartbeat`
- `deliver` : 是否送去 messaging channel
- `channel` , `to` : delivery destination

- `model`, `thinking` : override
- `timeout_seconds` : agent run timeout
- `request_timeout` : webhook HTTP timeout

## 實際 response

實測 webhook 成功回覆類似：

```
{
  "ok": true,
  "runId": "842d25f7-45c7-41c7-a97e-5cb98d0ec8c7",
  "status_code": 200
}
```

注意：唔一定有 `status: accepted` 。所以 code 應該用 `ok` 判斷 webhook 有冇成功。

### `call_mapped_hook()`

```
result = client.call_mapped_hook("gmail", {
  "source": "gmail",
  "messages": [
    {"from": "Ada", "subject": "Hello", "snippet": "Hi"}
  ]
})
```

用途：

- call 自定義 mapped hook

## 3. Tools invoke methods

### `invoke_tool()`

```
result = client.invoke_tool("sessions_list", {"limit": 10})
```

用途：

- call OpenClaw `/tools/invoke`

### `list_sessions()` / `list_sessions_details()`

```
raw = client.list_sessions(limit=10)
details = client.list_sessions_details(limit=10)
```

實測 raw format：

```
{
  "ok": true,
  "result": {
    "content": [...],
    "details": {
      "count": 3,
      "sessions": [...]
    }
  },
  "status_code": 200
}
```

平時建議直接用：

```
details = client.list_sessions_details(limit=10)
print(details["sessions"])
```

### `get_session_history()` / `get_session_history_details()`

```
raw = client.get_session_history("hook:main:test", limit=20)
details = client.get_session_history_details("hook:main:test", limit=20, in
```

實測 raw format：

```
{
  "ok": true,
  "result": {
    "content": [...],
    "details": {
      "sessionKey": "hook:main:test",
      "messages": [...]
    }
  },
  "status_code": 200
}
```

平時建議直接用：

```
details = client.get_session_history_details("hook:main:test", limit=20)
print(details["messages"])
```

## 4. call\_agent\_sync()

```
result = client.call_agent_sync(
    message="Please reply with exactly: SYNC_OK",
    agent_id="main",
    session_key="hook:main:test-sync",
    timeout=45,
    poll_interval=2,
)
```

用途：

- 先 call `/hooks/agent`
- 再 poll `sessions_history`
- 最後擱返 agent text reply

## 實測成功 response

```
{
  "status": "ok",
  "runId": "842d25f7-45c7-41c7-a97e-5cb98d0ec8c7",
  "reply": "SYNC_OK",
  "details": {
    "sessionKey": "hook:main:test-sync",
    "messages": [...]
  }
}
```

## implementation notes

`call_agent_sync()` 目前做法：

1. 先取 baseline history
2. call webhook
3. 再 poll `get_session_history_details()`
4. 只接受 baseline 之後新增嘅 assistant text message
5. 忽略純 `toolCall` block

## 5. content / message format 注意事項

`sessions_history` 裏面 `messages[*].content` 唔一定係 `string`，好多時係 block array，例如：

```
[
  {
    "type": "text",
    "text": "SYNC_OK"
  }
]
```

又或者：

```
[
  {
    "type": "toolCall",
    "id": "call_xxx",
    "name": "exec",
    "arguments": {...}
  }
]
```

所以：

- 唔可以假設 `content` 永遠係 plain string
- 要 normalize block content
- 做 sync polling 時要 skip 純 tool call message

## 6. `test.py` 用法

### 列 sessions

```
python3 test.py --sessions
```

### 查 session history

```
python3 test.py --history hook:main:test --limit 10
python3 test.py --history hook:main:test --limit 10 --include-tools
```

### wake test

```
python3 test.py --wake --message "Wake test from test.py"
```

## async webhook test

```
python3 test.py --async \  
  --message "Async test from test.py" \  
  --agent main \  
  --session-key hook:main:test-async \  
  --timeout 30
```

## sync webhook test

```
python3 test.py \  
  --message "Please reply with exactly: SYNC_OK" \  
  --agent main \  
  --session-key hook:main:test-sync \  
  --timeout 45 \  
  --poll-interval 2
```

## 7. 已完成實測

已經跑過以下測試：

- `wake()` ☒
- `list_sessions_details()` ☒
- `get_session_history_details()` ☒
- `call_agent()` async ☒
- `call_agent_sync()` sync ☒

### 特別修正過嘅 bug

#### Bug 1: tools invoke response schema 假設錯誤

原本假設 `messages` 直接係 `result.messages`，實際係係：

- `result.details.messages`
- `result.details.sessions`

## Bug 2: sync webhook success 判斷錯誤

原本以為 webhook 成功一定會有：

```
{ "status": "accepted" }
```

但實測成功回覆只係：

```
{ "ok": true, "runId": "..." }
```

所以而家改成用 `ok` 判斷。

## 8. 建議

平時 project code 建議：

- 取 sessions → 用 `list_sessions_details()`
- 取 history → 用 `get_session_history_details()`
- 想同步等 reply → 用 `call_agent_sync()`
- 想保留最原始 payload → 用 raw methods ( `list_sessions()` , `get_session_history()` , `invoke_tool()` )

## 9. 參考

- OpenClaw webhook docs:
  - <https://docs.openclaw.ai/automation/webhook.md>

如果之後 OpenClaw `/tools/invoke` response schema 再變，優先檢查：

- `result.details`
- `result.content[].text`
- `messages[*].content` block structure